

XJTLU Entrepreneur College (Taicang)

Technical Documentation — SynergyX

Module Code and Title	ENT208TC Industry Readiness
School Title	XJTLU Entrepreneur College (Taicang)
Assignment Title	Technical Documentation
Submission Deadline	Friday 15 May, 2026 before 23:59 (Beijing Time, GMT+8)
Session and Group	Session 2 — Group 18
Student IDs	2361293, 2362370, 2361552, 2362976, 2361394, 2360865, 2362921
Final Word Count	≈ 3,000 words (Sections 1–5)
Anonymous use for teaching and learning?	yes

I certify that I have read and understood the University's Policy for dealing with Plagiarism, Collusion, and the Fabrication of Data. My work does not contain any instances of plagiarism, collusion, or fabricated data.

By uploading my assignment onto Learning Mall Online, I formally declare that all of the above information is true to the best of my knowledge and belief.

Section 1 — System Architecture

1.1 Architecture overview

SynergyX is a mobile-first health tracking web application that combines manual logging, Apple Health data import, rule-based health scoring, and optional AI-generated guidance. The system follows a lightweight client-server model. The frontend is a React single-page application built with Vite and TypeScript that renders the interface, collects user input, maintains session state, stores most day-to-day health records locally, and invokes backend APIs for authentication and AI-backed features. The backend is an Express application that exposes REST-style endpoints for authentication, profile management, AI insight generation, nutrition analysis, and assistant chat. Persistent account data is stored in Supabase PostgreSQL, while runtime deployment targets Vercel through a serverless wrapper.

A key design decision is that identity and profile data are separated from daily tracking data. Account fields (name, email, height, weight, goal, activity level) are synchronised with the backend and stored in Supabase. Most operational records — meals, activity, sleep, water, weight, step imports, rhythm items, and plan tasks — are persisted on the client through Zustand stores. This reduces backend complexity and keeps the app responsive for a coursework prototype, with the cross-device and backup limitations discussed in Section 5.

1.2 System architecture diagram

SynergyX — System Architecture

Mobile-first health tracking web app • React SPA + Express API + Supabase + AI providers

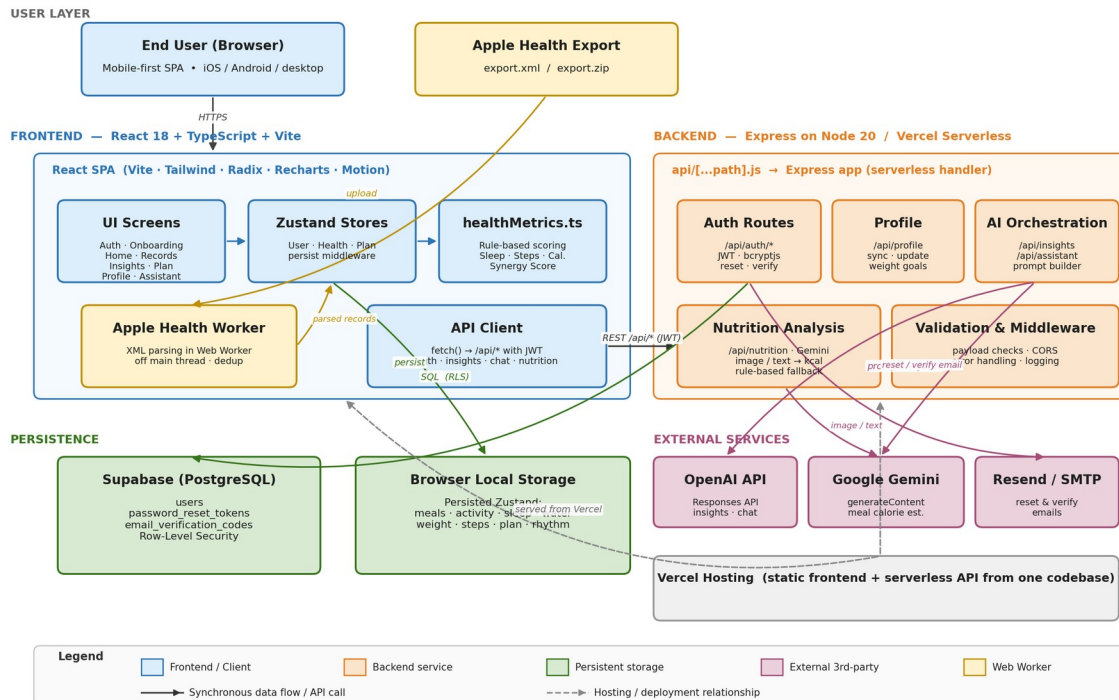


Figure 1. SynergyX high-level system architecture. Solid arrows show synchronous data flows; dashed arrows show hosting relationships.

1.3 Component overview

Frontend application. A React SPA containing the major screens — authentication, onboarding, home dashboard, record logging, insights, planning, profile, and the AI assistant modal — with lightweight state-driven routing and URL-path handling for flows such as password reset.

Stores. The user store manages authentication state, session validation, onboarding state, and profile fields. The health store persists activity, nutrition, sleep, water, weight, and imported step records via Zustand middleware, and performs deduplication during Apple Health import using a (type, timestamp, source, payload) signature. The plan store maintains rhythm items and daily plan tasks with daily-reset behaviour so recurring checklists are reused each day.

Apple Health import worker. Export files are parsed in a dedicated Web Worker to avoid blocking the main UI thread. The worker extracts supported XML records (sleep, body mass, workouts, steps), normalises units, filters by time window, and returns processed records for import.

Express backend, Supabase, and external services. The backend validates payloads, manages account data, constructs prompts for OpenAI and Gemini, and returns structured JSON. Supabase stores users, password_reset_tokens, and email_verification_codes under row-level security. Email

delivery for reset and verification is handled by Resend or SMTP behind an abstraction so the same flow works locally and on Vercel.

1.4 Data flow

After registration or login the backend hashes passwords with bcryptjs, issues a JWT, and reads or writes Supabase profile records; the frontend then renders the main tabs. Manual records are written directly into the local health store: an activity entry, for example, becomes a record holding subtype, duration, optional distance, and estimated calories. Apple Health imports are parsed in a Web Worker, deduplicated, and merged into the store.

The Home, Insights, and Plan screens compute summary metrics — sleep duration, activity minutes, total calories, step totals, weight trend, and a composite synergy score — from the local stores using rule-based functions in healthMetrics.ts. For richer analysis, the frontend posts a summarised payload to /api/insights/analyze or /api/assistant/chat. The backend constructs prompts from the user profile, recent records, rule-based summaries, and current plan items, then returns structured JSON. This grounding reduces hallucination compared with an unconstrained chatbot.

1.5 Technology stack summary

Frontend: React 18, TypeScript, Vite 6, Tailwind CSS, Radix UI, Recharts, Motion, Zustand. Backend: Node.js 20, Express 5, Supabase, JWT auth, bcryptjs, Resend/SMTP. AI: OpenAI Responses API and Google Gemini generateContent. Deployment: Vercel, with api/[...path].js wrapping Express as a serverless handler.

Section 2 — Technology Justification

For every key technology we considered at least one credible alternative. The table below states what was chosen, what was considered, and the specific trade-off driving each decision.

Technology / Tool	Chosen	Alternatives considered	Why we chose this
Frontend framework	React 18 + TypeScript	Vue 3, Next.js, plain JS SPA	Mature component model and direct compatibility with the Radix UI and Recharts libraries we use. TypeScript made records and API payloads explicit. Next.js was rejected because the app needs no SSR; a lighter SPA reduced overhead.
Build tool	Vite 6	Create React App, Webpack	Near-instant hot reload via

			native ES modules and a simple config surface. Fast feedback mattered more than Webpack's customisation; CRA is no longer actively maintained.
State management	Zustand + persist middleware	Redux Toolkit, React Context	Tiny (~1 kB), modular stores, and persist middleware makes local-first storage trivial. Redux adds boilerplate we do not need; Context re-renders too aggressively for high-frequency updates.
Backend framework	Express on Node 20	Fastify, NestJS, Supabase Edge only	Minimal ceremony surface, integrates cleanly with both local Node and the Vercel serverless wrapper. NestJS adds DI/decorators we do not need; Edge alone would have split auth and AI logic across runtimes.
Database / hosted backend	Supabase (PostgreSQL)	Firebase, MongoDB Atlas, local SQLite	Managed PostgreSQL, row-level security, and a service-role key model that matches our auth flow. Relational tables fit our schema better than Firestore documents; SQLite alone cannot support multi-user remote auth.
AI integration	OpenAI Responses + Google Gemini	Single-provider AI, no AI features	Two providers reduce vendor lock-in and allow meal-image analysis via Gemini while still using OpenAI for richer chat. A rule-based fallback preserves baseline functionality when either API is unavailable.
Deployment target	Vercel + serverless API	Render, Railway, self-hosted VPS	Matches our static-frontend-plus-API model and deploys from one codebase. The serverless wrapper avoids paying for an always-on container

			during a coursework demo, unlike a VPS we would have to operate ourselves.
--	--	--	--

2.2 Trade-offs and technical risks

The main trade-off is storing most health records on the client while keeping authentication and profile data on the server. This reduced backend complexity and accelerated implementation but loses automatic cross-device sync and robust cloud backup; Apple Health import can partially re-seed device data after a store reset, which made the trade acceptable for a coursework scope. A second risk is reliance on third-party AI APIs: models can fail, lag, or drift in wording. We mitigated this with JSON-constrained responses, prompts grounded in app data, per-request timeouts, and rule-based local fallbacks. The Apple Health importer also adds parsing risk because export files can exceed 50 MB of XML; we mitigated this by streaming parsing inside a Web Worker and limiting imports by time window. Finally, the Supabase service-role key is kept strictly server-side and complemented by row-level security so a leaked anon key alone cannot expose other users' data.

Section 3 — Deployment Guide

3.1 Environment requirements

Requirement	Version / Notes
Node.js	20.x (declared in package.json engines)
npm	≥ 10.x, bundled with Node 20
Supabase project	Required for users, password_reset_tokens, email_verification_codes (schema in /supabase)
AI provider key	At least one of OPENAI_API_KEY or GEMINI_API_KEY; rule-based fallback works without either
Email provider (optional)	Resend API key, or SMTP credentials, for password reset and verification email

3.2 Setup steps

1. Clone the repository: `git clone https://github.com/<your-org>/synergyx.git && cd synergyx`.
2. Install dependencies: `npm install` (installs both frontend and backend packages declared in package.json).
3. Configure environment variables: `cp .env.example .env`, then set the required server variables (APP_BASE_URL, JWT_SECRET, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY) and at least one AI key (GEMINI_API_KEY and/or OPENAI_API_KEY). Use `ENABLE_EMAIL_VERIFICATION=false` for initial

testing. On Vercel, configure the same variables under Project Settings → Environment Variables, and apply Supabase migrations from /supabase/migrations before first run.

4. Run the application. Locally, in two terminals: npm run server (Express API on :3000) and npm run dev (Vite client on :5173). For production: npm run build, then vercel deploy --prod.

5. Verify it is working. Open <http://localhost:5173> (or the Vercel preview URL) and confirm that (a) the onboarding screen renders without console errors, (b) registration and login succeed, (c) the Home dashboard shows zeroed metrics, (d) adding a manual activity record updates the dashboard immediately, and (e) the Insights screen returns AI text within ~10 s when an AI key is set, or a rule-based summary otherwise. Calls to /api/auth/login, /api/profile, and /api/insights/analyze should each return HTTP 200 with JSON.

3.3 Common issues and solutions

Problem	Likely cause	Solution
Login/registration fails with server configuration error	SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, or JWT_SECRET missing	Re-check backend env vars, ensure no trailing whitespace, restart npm run server and re-deploy on Vercel.
AI insights or assistant do not respond	No / invalid OPENAI_API_KEY or GEMINI_API_KEY, or blocked outbound network	Add a valid key, verify outbound access, check server logs for 401/429. Rule-based summary should still render as fallback.
Password reset / verification email never arrives	Email provider variables missing or sender domain not verified	Set ENABLE_EMAIL_VERIFICATION=false for basic testing, or configure RESEND_API_KEY and EMAIL_FROM, with verified domain SPF/DKIM, before enabling mail flows.
Apple Health import seems slow or fails silently	Wrong file selected, or export contains unsupported record types	Use the official export.xml / export.zip; test a 7-day window first, then widen to 30 d or all.

Section 4 — IP Strategy

This section is a learning exercise in strategic thinking, not legal advice.

4.1 Novelty analysis

We do not claim novelty in logging meals, sleep, or steps. What is novel is the specific combination of (a) a local-first client store that keeps daily logs in the browser through persisted Zustand, (b) an Apple Health export importer that runs in a Web Worker and deduplicates by record signature, (c) a rule-based synergy score computed entirely on the device, and (d) an AI orchestration layer that grounds OpenAI and Gemini prompts in this structured local state rather than free-form user prose.

Compared with Apple Health, Fitbit, or MyFitnessPal, the result is a hardware-independent, privacy-leaning prototype that produces explanations scoped to the user's own records. The novelty is at the system-design and grounding-strategy level, not at the algorithmic level.

4.2 Prior art

Existing product	What it does	How SynergyX differs
Apple Health	Aggregates wellness and device data in the Apple ecosystem; allows export of health records as XML.	SynergyX does not replace Apple Health as a data platform; it consumes the exported XML and re-purposes it inside a planning and AI-explanation workflow for student wellness.
Fitbit app	Tracks activity, sleep, and metrics with dashboards, deeply integrated with Fitbit hardware and cloud sync.	Hardware-independent, supports manual records alongside imports, and emphasises local-first storage plus AI-generated text rather than device-ecosystem lock-in.
MyFitnessPal	Meal logging, calorie tracking, weight management with a large commercial food database.	Adds Apple Health import, daily habit planning, and a contextual AI assistant grounded in user records, but on a smaller technical scope.

4.3 Patent decision

No — we do not recommend filing a patent at this stage. The technical novelty is an integration of well-known building blocks (Web Workers, persisted client stores, JWT auth, LLM prompting) rather than a non-obvious algorithm, making the inventive-step bar (US 35 U.S.C. §103, EPO Article 56) hard to meet, and software patents covering AI-augmented UIs are increasingly challenged as abstract under Alice v. CLS Bank. Filing typically costs USD 8–15k per jurisdiction, takes several years, and is only useful if the owner can afford to enforce it — which a seven-person student team cannot. The pragmatic path is to publish the architecture (creating defensive prior art) and rely on faster non-patent protections.

4.4 Alternative protection

First-mover advantage and trust. Health apps depend on user trust; documenting our local-first storage, row-level security, and constrained-JSON prompting credibly differentiates the product on privacy and is hard for competitors to copy quickly. Trade secrets on prompts and scoring. The prompt templates used for /api/insights/analyze and the weights in healthMetrics.ts are not visible from the compiled frontend bundle; they can be kept server-side and reinforced with NDAs and access-controlled repositories. Copyright, licensing, and trademark. Source code is automatically

copyrighted, and choosing a deliberately restrictive licence (e.g. AGPL-3.0 or BSL) deters direct commercial forks. Trademarking “SynergyX” is comparatively cheap (~USD 250–350 per class) and protects the brand even when the underlying technology cannot be patented. Speed and network effects. Iterating quickly and accumulating grounded health-Q&A pairs creates a data and UX moat — typically a stronger defence in software than a single patent claim.

Section 5 — Limitations & Future Work

5.1 Known limitations

Limitation	Why it exists	Impact on users
Most health and plan data live only in browser storage	Persisted Zustand for speed and simplicity instead of a full cloud-sync backend.	Records can be lost if browser storage is cleared, and do not automatically sync across devices.
AI output quality depends on external APIs and prompt grounding	Insights and assistant replies rely on OpenAI / Gemini availability and response quality.	Latency, occasional failure, or uneven wording — especially when keys are missing or rate-limited.
Apple Health import covers only selected record types	Parser focuses on sleep, body mass, workouts, and step counts.	Richer signals (HRV, blood oxygen, menstrual cycle) cannot yet be imported.
No mobile shell or native push notifications	The product is a web SPA, not a wrapped iOS/Android binary.	Habit reminders depend on the user re-opening the tab, weakening daily-engagement loops.

5.2 Future work

Planned features. (i) Optional cloud sync that migrates the health, plan, and rhythm stores to Supabase behind a per-user feature flag so existing local-first behaviour is preserved; (ii) extended Apple Health coverage adding heart rate, HRV, blood oxygen, and active energy into the daily aggregation pipeline; (iii) a Capacitor or PWA-installable shell to enable home-screen install and background notifications; (iv) server-sent-event streaming for /api/insights and /api/assistant so long responses feel faster.

Technical debt to address. Replace ad-hoc payload checks in the Express routes with a schema validator (Zod) so contract drift is caught at compile time; add unit tests for healthMetrics.ts and the Apple Health parser, plus Playwright end-to-end tests for auth and onboarding; and add request-level logging plus error reporting (e.g. Sentry) so AI provider outages and slow imports stop being silent.

Open research questions. (i) Does grounding LLM responses in user-specific structured state measurably reduce hallucination for wellness use cases where ground truth is subjective? (ii) What

is the right balance between local-first privacy and cloud-backed convenience for student wellness apps, and how should users be informed of the trade-off during onboarding? (iii) Can a small rule-based scoring function be validated against published lifestyle-medicine guidelines without clinical trials, and what disclaimers should accompany it?

References

References follow APA 7th edition style. Software documentation is cited as a software/manual entry; retrieval date 14 May 2026.

Alice Corp. Pty. Ltd. v. CLS Bank International, 573 U.S. 208 (2014).

Apple Inc. (2025). Export your health data from the Health app on iPhone. Apple Support.

<https://support.apple.com/guide/iphone/share-your-health-data>

European Patent Office. (2024). Guidelines for examination, Part G, Chapter VII: Inventive step (Article 56 EPC). <https://www.epo.org/legal/guidelines-epc.html>

Express. (2025). Express — Node.js web application framework (v5.1) [Software documentation]. OpenJS Foundation. <https://expressjs.com/>

Google. (2025). Gemini API: generateContent reference [Software documentation]. <https://ai.google.dev/api/generate-content>

Meta Open Source. (2024). React documentation. <https://react.dev/>

OpenAI. (2025). Responses API reference [Software documentation]. <https://platform.openai.com/docs/api-reference/responses>

Pmndrs. (2024). Zustand: A small, fast and scalable bear-bones state-management solution (v5) [Software]. GitHub. <https://github.com/pmndrs/zustand>

Supabase Inc. (2025). Supabase documentation: Auth, Postgres, and row-level security. <https://supabase.com/docs>

U.S. Patent Act, 35 U.S.C. § 103 — Conditions for patentability; non-obvious subject matter (2025).

Vercel Inc. (2025). Deploying Node.js APIs as serverless functions. <https://vercel.com/docs/functions>

Vite. (2025). Vite 6 documentation [Software documentation]. <https://vitejs.dev/>

World Wide Web Consortium. (2023). Web workers, level 1 (W3C Working Draft). <https://www.w3.org/TR/workers/>